

FRUGAL TESTING

AI-Native Software Engineer Intern

Submitted by:

Sonika Deshwal

Lovely Professional University

Registration No: 12312987

Batch Year: 2027 | Drive Code: OC.21365.2027.61834

Date: June 28, 2026

Section 0: Pre-Evaluation Disclosures & Alignment

1. Bond Consent

Yes, I confirm my consent to sign the 36-month bond including the 12-month internship period, provided the terms and compensation details are as communicated by the placement coordinator.

2. Assignment Submission Confirmation

Yes, I am comfortable with the bond terms and am submitting this assignment accordingly.

3. Stipend and CTC Details

Yes, I have received the stipend and CTC details. Stipend: Rs. 25,000 per month during the 12-month internship period, followed by a full-time CTC of Rs. 12 LPA upon conversion. Drive Code: OC.21365.2027.61834 | Company Code: 21365.

4. Relocation to Gachibowli, Hyderabad

Yes, I am willing to relocate to Gachibowli, Hyderabad for this role.

5. Motivation for AI-Native Software Engineering

I have always been drawn to the intersection of software quality and intelligence systems. What excites me about AI-Native Engineering is that it is not just about writing tests — it is about building systems that reason about correctness. During my final year, I worked on a full-stack CRM project where I integrated LLM-driven features and quickly realized that the hardest problems were not in generating code — they were in validating it, catching edge cases AI tools miss, and building trust in autonomous output. That gap between what AI generates and what is actually production-safe is exactly where I want to specialize.

6. Why Frugal Testing

Frugal Testing sits at a genuinely interesting inflection point — the testing industry is being disrupted by the same AI tools it is supposed to validate. I want to be part of an organization that is actively engineering the response to that disruption rather than just reacting to it. The technical depth of this assignment confirmed that Frugal Testing approaches quality engineering as a systems problem, not a checkbox exercise — which is the environment I want to grow in.

Section A: Practical Anti-AI Engineering & Automation

All source code, execution videos, and workflow walkthroughs are hosted in Google Drive folders linked below. Each folder contains complete raw source code scripts and a single sequentially combined workflow video showing (1) output window, (2) source code walkthrough, and (3) GenAI prompt history.

Q1. Dynamic HTML5 Canvas State Drifts & Asynchronous Race Interceptions

Google	Drive	Folder	Link:
			https://drive.google.com/drive/folders/114IfRaSvIMnkSNWtBB6-bhhGIPAWGQU8?usp=sharing

Folder Contents: canvas_server.js (WebSocket testbed server), canvas_page.html (HTML5 Canvas streaming page), test_q1_canvas.py (Playwright automation covering all 4 execution specs), workflow video.

Q2. Cryptographic Replay Testing, Stateful Nonces & Hash-Chain API Chaining

Google	Drive	Folder	Link:
			https://drive.google.com/drive/folders/1iua0AcU4pvmwL0ZCFKzGfhPRwyEUmn0e?usp=sharing

Folder Contents: mock_api_server.js (Express server with HMAC-SHA512 nonce validation and replay protection), test_q2_replay.py (dynamic sequence chaining, HMAC generation, replay attack vector, assertion layer), workflow video.

Q3. Sealed Closed-Boundary Shadow DOM Pathfinding & Accessibility Tree Refactoring

Google	Drive	Folder	Link:
			https://drive.google.com/drive/folders/1hqhooVDxB-MLldkQnGa0JvUZZbV76iwU?usp=sharing

Folder Contents: q3_shadow_dom_and_cot.py (JavaScript recursive Shadow DOM traversal engine + Chain-of-Thought CoT system prompt for OS Accessibility Tree navigation), workflow video.

Section B: Core Competencies, AI Reasoning & Scenarios

Author: Sonika Deshwal | Lovely Professional University | Reg: 12312987

Q4. Architectural Critique: The Cascading Drift in Multi-Agent Synthesis Pipelines

Answer 1 — Structural Vulnerability

The core vulnerability is dependency mirroring — Agent B derives expected behavior exclusively from Agent A's already-corrupted output. When Agent A introduces a race condition or data isolation flaw, it becomes the de facto specification for Agent B's assertion layer. Agent B then encodes the flawed state as the correct baseline, and Agent C — evaluating only pass/fail metrics — issues a green deployment signal. No agent in this chain holds an independent ground truth. The false-positive approval chain emerges because each agent trusts its upstream peer rather than a static, immutable specification.

Answer 2 — External Deterministic Validation Layer

Introduce a Specification Compliance Gate between Agent B and Agent C — a non-generative, rule-based engine that validates behavior against a human-authored, version-controlled contract file (e.g., OpenAPI specs, JSON Schema assertions, or formal property files). This gate never reads Agent A's modified source — it only compares runtime outputs against the frozen specification baseline. Tools like Pact for contract testing or Allure's assertion layer with static rule sets work well here.

Q5. Log File Analysis: Garbage Collection Leaks & Microtask Loop Starvation

Answer 1 — Event Sequence to V8 Collapse

The crash follows a clear cascade. Socket descriptor 12 overflows its buffer — incoming data arrives faster than the microtask queue can drain it. This creates 68,240 unresolved closures inside the stream aggregator, each holding live heap references. The GC compaction cycle fires every 12ms trying to reclaim memory, but closures prevent object collection. Heap allocation hits 98.4%, new allocations begin failing, and V8 triggers FatalProcessOutOfMemory, halting the Node.js process entirely.

Answer 2 — Why UI Automation Returns Green

Standard end-to-end UI tests execute single-user flows in low-concurrency conditions — they never saturate the socket buffer or trigger microtask queue backlog. Functional assertions check DOM state and HTTP responses, not memory telemetry or GC compaction frequency. The crash only manifests under parallel, high-throughput load — conditions that UI automation frameworks do not replicate.

Q6. AI Code Safety Review & Prompt Engineering Mitigation

Answer 1 — SQL Injection / Tenant Isolation Breach

The function uses Python f-string interpolation to build raw SQL, directly embedding `tenant_id`, `target_metric`, and `filtering_date` without parameterization. A malicious actor can inject SQL through any of these parameters. Passing `tenant_id = '1 OR 1=1'` turns the WHERE clause into a universally true condition, returning all tenants' records. A UNION SELECT payload directly breaks tenant isolation, exposing adjacent data spaces across the entire `analytics_records` table.

Answer 2 — Rewritten Developer System Prompt

You are a secure database query generation engine. Rules:

1. NEVER use string interpolation or f-strings to build SQL.
2. ALWAYS use parameterized queries with placeholders (%s).
3. Signature: (tenant_id:int, target_metric:str, filtering_date:str, connection_pool:object) -> list
4. Query: cursor.execute("SELECT * FROM analytics_records WHERE tenant_owner=%s AND metric=%s AND processed_at>=%s", (tenant_id, target_metric, filtering_date))
5. Validate input types - raise TypeError on mismatch.
6. Return cursor.fetchall() only.
7. Output only the function code block.

Q7. Flaky Test Code Review & Clock-Drift Desynchronization

Answer 1 — Why This Flakes in Cloud Runners

The 15,000ms setTimeout is a static delay assuming consistent clock behavior and single-tenant CPU availability. In shared-core GitHub Actions or AWS CodeBuild runners, the container CPU is time-sliced across multiple concurrent jobs causing clock-drift. The event loop may stall longer than expected under CPU contention, so the toast element resolves before or after the static delay window unpredictably. The page.reload() in the else branch also creates a non-deterministic second execution path.

Answer 2 — Refactored Deterministic Version

```
await page.goto("https://core-platform.com/ledger-vault");
await page.waitForSelector(".transaction-complete-toast", {
  state: "visible", timeout: 30000
});
await page.locator("#action-confirm-btn").waitFor({ state: "visible" });
await page.locator("#action-confirm-btn").click();
```

Q8. Systems Concurrency & Connection Pool Leak Mechanics

Answer 1 — Profiling Strategy

Enable HikariCP JMX metrics and export to Grafana + Prometheus. Run the 200-browser test suite while capturing: active connection count, idle connection count, connection acquisition wait time P95/P99, and thread state dumps via jstack. If active connections saturate the pool maximum and wait times spike simultaneously, the cause is pool size misconfiguration. If thread dumps show threads in BLOCKED state on database locks, the root cause is nested DB lock contention.

Answer 2 — Key Telemetry Metrics

hikaricp_connections_active (compare against maximumPoolSize), hikaricp_connections_pending (threads waiting), hikaricp_connection_acquisition_ms P95/P99 (latency spike confirms starvation), hikaricp_connections_timeout_total, Thread State breakdown RUNNABLE vs BLOCKED vs WAITING from jstack, DB lock wait events from pg_locks (PostgreSQL) or INNODB_LOCKS (MySQL).

Q9. Operational Ambiguity: Headless CSS Layout Tree Thread Collapses

Answer 1 — Why Automation Passes Despite Blank Screen

Functional automation validates HTTP response codes, DOM element presence, and JavaScript event completions. A CSS-in-JS compilation crash occurs post-DOM-construction — the HTML shell exists and elements are technically present in the DOM tree, but the layout engine fails to compute visual rendering coordinates. Playwright's isVisible() checks only confirm DOM attachment and CSS display property

values — they do not validate that the layout tree was successfully painted to the viewport.

Answer 2 — Visual Triage Strategy

Implement a multi-layer visual validation: (1) Layout paint metrics — assert first-contentful-paint exists with non-zero startTime via `performance.getEntriesByType('paint')`. (2) Console error extraction — capture `page.on('console')` events and fail tests if ERROR entries relate to CSS-in-JS runtime crashes. (3) Pixel-level screenshot diff — run baseline visual regression using Playwright's `expect(page).toHaveScreenshot()` to catch blank viewport states.

Q10. Next-Generation Agentic Loops: Autonomous Multi-Branch Cascading Loops

Answer 1 — External Architectural Validation Layer

Design an Agent Control Plane between the agent and Git/cloud APIs: (1) Write quota enforcer — limit agent to maximum 3 branch creations per 10-minute window via token-bucket rate limiter. (2) Sandbox isolation — agent writes restricted to ephemeral container; human-approval webhook required before any PR merge. (3) Revert authority — control plane maintains rollback snapshot of pre-agent repository state. (4) Compute budget guard — hard ceiling on cloud API calls per session.

Answer 2 — Agentic Hallucination Loop Telemetry

Flag a hallucination loop when ALL breach thresholds simultaneously: branch creation rate greater than 3 per 5 minutes, unique test failure messages not decreasing across consecutive retry cycles, semantic similarity score between consecutive git commit messages greater than 0.92, cloud compute spend velocity greater than 2x the rolling 7-day average, time-since-last-human-approval greater than 30 minutes.

Q11. AST-Driven Test Selection Frameworks & Contextual Path Dependency Mapping

Answer 1 — AST Diff Parsing Logic

On each pull request, the framework parses the incoming diff using an AST parser (tree-sitter or Python's `ast` module) to extract modified function signatures, class methods, and module imports. It then traverses a pre-built dependency graph where each node is a code unit and each edge represents a call or import relationship. Any test file whose execution path intersects a modified node is marked as impacted. Only impacted tests are scheduled for execution.

Answer 2 — Preventing Critical Coverage Gaps

The framework maintains a coverage heat map from historical full-suite runs tagging each test with the integration paths it exercises. For any modified module, the framework additionally selects all tests covering downstream consumers — not just direct callers. A mandatory boundary suite covering authentication, payment, and data isolation paths always runs regardless of diff scope.

Q12. Self-Healing Testing Engines: Graph-Based Structural Neighbor Analysis

Answer 1 — Algorithmic Failures

The self-healing engine failed on two levels. First, similarity scoring used only surface-level string distance between `#confirm-balance-wipe` and `.btn-danger` — semantically unrelated but the engine had no semantic weight layer. Second, the engine had no destructive action flag system — it treated a

database-wide button identically to a modal close button. Proximity in the DOM tree does not imply functional equivalence.

Answer 2 — Confirmation Protocol

Build a composite confidence score: (1) Levenshtein distance — penalize scores above 5 edits. (2) Graph neighbor score — count shared parent nodes, sibling count, depth delta in DOM tree. (3) Semantic role weight — assign danger multiplier to elements with data-action=delete or destructive aria-labels. (4) Only auto-heal if final score above 0.85 AND danger multiplier equals 0. If danger multiplier above 0, halt and raise human-review alert.

Q13. Model Context Protocol (MCP) Sandboxing: Zero-Trust Schema Configurations

```
{
  "name": "read_system_logs",
  "description": "Read-only: trailing 150 lines within /var/app/logs/ only.",
  "input_schema": {
    "type": "object",
    "properties": {
      "log_filename": {
        "type": "string",
        "pattern": "^[a-zA-Z0-9_\\-]+\\.log$",
        "maxLength": 64
      }
    }
  },
  "required": ["log_filename"],
  "additionalProperties": false
},
"execution_policy": {
  "allowed_base_path": "/var/app/logs/",
  "command_template": "tail -n 150 /var/app/logs/{log_filename}",
  "forbidden_patterns": ["&&", "|", ";", "|", ">", "$(", "rm", "curl"],
  "read_only": true,
  "max_output_lines": 150,
  "shell_expansion_disabled": true
}
}
```

Q14. Systems Scalability: Asynchronous Log Ingestion Topographies

Answer 1 — Horizontally Scalable Architecture

Incoming payloads hit a lightweight API gateway that immediately writes each request to a message broker (Apache Kafka or AWS SQS) and returns HTTP 202 Accepted. Kafka partitions payloads by tenant or branch ID. A pool of background worker nodes (Celery + Redis or AWS Lambda) pulls from Kafka, decompresses base64 screenshots, extracts stack traces, and writes to tiered storage: hot metadata to PostgreSQL, raw screenshots to S3. Worker pool scales horizontally based on Kafka consumer lag metrics.

Answer 2 — Protecting Downstream Rate Limits

LLM token rate limiting — implement a token-bucket queue in front of the LLM API call; overflow requests queued with exponential backoff. DB connection protection — use PgBouncer as connection pooler; pool size capped at 80% of max DB connections. Priority lanes — critical failure payloads (P0 severity) skip the queue and get dedicated worker threads; verbose info logs batched and processed off-peak.

Q15. Distributed Tracing & Cascade Failures across Distributed Ledgers

Answer 1 — Root Cause Isolation

The failure originates at Span 5: LedgerDB UPDATE — a row-level lock wait timeout (2043ms) on `user_accounts` WHERE `id=92`. Span 3 (LedgerEngine: ExecuteTransfer) inherits this ERROR since it was blocked waiting for Span 5 to commit. Span 1 (API Gateway) surfaces the final HTTP 500 because its child span never resolved. Span 2 (TokenService) and Span 4 (AccountService) completed successfully — confirming the failure is isolated to the LedgerDB write layer.

Answer 2 — Distributed Correlation Tracking

OpenTelemetry propagates a traceparent header (W3C Trace Context standard) across every service boundary. Each span records its `trace_id`, `span_id`, and `parent_span_id`. When LedgerDB generates Span 5, it inherits the same `trace_id` as Span 1 — allowing the full causal chain to be reconstructed in any APM tool by filtering on a single `trace_id` value.

Answer 3 — Database Team Triage Briefing

TO: Database Platform Team | RE: LedgerDB Lock Contention — Row ID 92

Root cause: Lock Wait Timeout on `user_accounts` row `id=92` during concurrent checkout load simulation. Required Actions: (1) Set transaction isolation level to READ COMMITTED on LedgerEngine DB connection. (2) Audit open transactions using `SELECT * FROM pg_locks JOIN pg_stat_activity`. (3) Add index on `user_accounts(id, active_balance)`. (4) Set `lock_timeout=5000ms` at session level. (5) Evaluate whether AuditBalances read should run on a separate read-replica to remove read/write contention on primary.

Q16. Cognitive Prompt Critiques: Halting the Context Contraction in Refinement Cycles

Answer 1 — Architectural Flaws

The developer is using a progressive refinement pattern where each turn adds a new constraint without a complete problem specification upfront. The LLM builds each response on top of previous partial context — by turn 3, the model is extending an already-broken regex rather than redesigning from a clean specification. Each iteration narrows the model's generative space while adding conflicting constraints. The context window is consumed by failed attempts, leaving less effective space for actual reasoning.

Answer 2 — Single High-Fidelity CoT Prompt

You are a regex engineering expert. Generate a single Python regex pattern satisfying ALL constraints simultaneously.

INPUT: Extract complete JSON objects from unstructured log files.

Log format: "2026-01-15T14:32:01.123Z [LEVEL] <content>"

JSON may span multiple lines with deeply nested arrays/objects.

CONSTRAINTS:

1. Use `re.DOTALL` flag for multiline spans
2. Match AFTER the log prefix (timestamp + level tag)
3. Handle balanced braces - no partial JSON fragments
4. Handle nested arrays correctly
5. Do not match empty objects `{}`

OUTPUT: Return ONLY the raw regex pattern string.

Q17. Quality Engineering Blueprint: Critical Infrastructure Data Flow Distortions

Answer 1 — Resource Allocation Across Testing Tiers

For a HIPAA-compliant healthcare wearable data platform: Unit Testing (20%) — automated, developer-owned. Application Security (30%) — highest priority; covers OWASP Top 10, PHI data exposure, encryption validation. API Functional (20%) — validates device data ingestion contracts. Consumer-Driven Contract (10%) — ensures wearable device SDK and portal API remain backward-compatible. Multi-Modal Visual Regression (10%) — catches UI anomalies in patient-facing dashboards. Load Testing (10%) — validates system durability during peak data ingestion spikes.

Answer 2 — Non-Overlapping Tier Roles

Unit — validates individual data transformation functions in isolation. App Security — identifies PHI exposure vectors, broken access controls, insecure data transmission. Consumer-Driven Contract — validates device firmware SDK and portal backend agree on data schema. API Functional — end-to-end validation of ingestion pipelines. Visual Regression — catches rendering anomalies not detectable via API assertions. Load — validates connection pool behavior, DB write throughput, and queue saturation under thousands of simultaneous device streams.

Q18. OpenAPI Specification Boundary Exploitation & Semantic Attack Topographies

Answer 1 — Boundary-Value & Negative Test Vectors

tenantId: 999 (below min 1000), 1000000 (above max 999999), 0, -1, 1000.5 (float), '1001' (string coercion). transactionAmount: 0 (below min 0.01), 50000.01 (above max), -100.00 (negative), null, string, 1e+308 (float overflow). accountPasscode: '123' (below minLength 4), '123456789' (above maxLength 8), 'ABCD' (non-numeric), SQL injection string. X-Idempotency-Key: missing, malformed UUID, reused UUID. targetRegion: outside enum, null, array injection. metadataPayload: deeply recursive nested object (stack overflow vector).

Answer 2 — Input Verification & Assertion Parameters

Assert HTTP 400 for all type-coercion bypass attempts. Assert HTTP 422 for enum violations on targetRegion. Assert HTTP 409 for duplicate X-Idempotency-Key reuse. Assert response schema strictly matches OpenAPI definition. Assert tenantId values outside [1000, 999999] never reach database layer. Assert deeply nested metadataPayload beyond 10 recursion levels returns HTTP 413 or 400. Assert no 500 errors returned for any input combination.

Q19. Automated Quality Release Sign-Off Gates

Answer 1 — AI-Native Sign-Off Gate Architecture

The gate sits at the end of the CI pipeline as a blocking step. It ingests metrics from four data sources via API: code coverage reports (Istanbul/JaCoCo), test execution history (TestRail/Allure), container security scans (Snyk/Trivy), and Jira bug tracker. A rules engine processes these inputs against pre-configured thresholds and outputs a Go/No-Go decision.

Answer 2 — Metric Ingestion & Weighting

Statement coverage $\geq 85\%$ and branch coverage $\geq 80\%$ — required gate (hard block if unmet). Integration test pass rate $\geq 98\%$ over last 3 runs — weighted score. Zero Critical/High CVEs in container scan — required gate. No P1/P0 Jira bugs in Open status — required gate. P2 bug count ≤ 5 — weighted score. Weighted score ≥ 90 auto-approves. Score 70-89 triggers human review. Below 70 or any

hard-block failure triggers automated rollback.

Q20. Closed-Loop Observability: Adaptive Production-Driven Stress Testing

Answer 1 — Linking Production Metrics to Pre-Deployment Suites

Production APM tools (Datadog, New Relic) expose error rate, latency P99, and throughput metrics per endpoint via API. A bridge service polls these metrics every 5 minutes and maps them to the test suite's coverage registry. When a production endpoint's error rate exceeds a configurable threshold (greater than 2%), the bridge automatically flags the corresponding test scenarios as high-priority and triggers a targeted re-run against staging with enhanced logging.

Answer 2 — Adaptive Chaos Injection Setup

The framework subscribes to production traffic telemetry via a streaming API. A traffic analyzer module identifies the top 5 service paths by current request volume in real time. The test orchestrator dynamically scales up execution nodes via Kubernetes HPA and launches chaos-injection scenarios targeting those high-traffic paths — simulating latency spikes, connection drops, and payload corruption on the exact service channels users are actively hitting.

Behavioral & Fit Evaluation: Psychological Alignment Profiles

Situation A: The Undocumented Legacy Crash

My Choice: Choice ii

With 4 hours to release, business continuity takes priority. I would wrap the failing module in a structured error boundary to stabilize the pipeline immediately, document my observations, and create a tracked Jira ticket for a proper post-launch refactor. Speed without recklessness — the wrapper is a deliberate engineering decision, not a hack.

Situation B: Autonomous AI Agent Alignment Conflicts

My Choice: Choice i

Performant code that violates team conventions creates long-term maintenance debt. I would pause the loop, audit the system prompt, and embed explicit structural linting constraints. Short-term velocity loss is worth it — consistency in a shared codebase is a force multiplier for the whole team.

Situation C: Technical Ambiguity vs. Speed to Market

My Choice: Choice ii

When requirements are genuinely ambiguous and changing daily, over-planning burns time on assumptions that will shift anyway. I would ship a minimal working prototype quickly, use real execution failures as feedback signals, and iterate. This is using the system itself as the specification discovery tool.

Situation D: The Code-Coverage Metric Divergence

My Choice: Choice i

Coverage percentage is a proxy metric, not a quality guarantee. Writing hollow tests to hit 95% is technical dishonesty — it creates false confidence. I would raise this directly with the engineering director and advocate for mutation testing or contract test frameworks that measure test effectiveness, not just line count.

Q21. Professional Technical Article

Securing the AI Workspace: Designing Restrictive MCP Sandboxes to Prevent Arbitrary Code Executions by Autonomous Developer Agents

By Sonika Deshwal | Lovely Professional University | Reg: 12312987

Introduction

The rise of autonomous developer agents — AI systems capable of reading codebases, writing files, executing shell commands, and triggering CI/CD pipelines — has fundamentally changed what it means to secure a software engineering environment. These agents do not just generate suggestions; they act. And when they act without architectural boundaries, the consequences range from accidental data deletion to full system compromise. The Model Context Protocol (MCP) has accelerated this capability by making it trivially easy to give an AI agent access to a terminal, a database, a file system, or an API gateway. What it does not provide out of the box is any meaningful restriction on what the agent can do with that access.

Why Default MCP Tool Schemas Are a Security Liability

A standard MCP tool schema that grants an agent shell access with an unescaped command string is effectively a root shell handed to the agent with no guardrails. The agent can execute destructive commands, exfiltrate secrets, chain commands using `&&` or `||`, spawn subshells, redirect output to arbitrary file paths, or install packages. The danger compounds when LLMs hallucinate — confidently executing commands that seem logically correct to the model but are semantically destructive in the actual environment.

The Zero-Trust Design Philosophy for MCP Sandboxes

Zero-trust applied to MCP environments means a single principle: the agent is never trusted by default, regardless of its instruction source. Every tool call must be evaluated against an explicit allowlist of permitted operations, not a denylist of forbidden ones. Denylists fail open — they block what you have thought of and permit everything else. Allowlists fail closed — they permit only what has been explicitly approved.

Layer 1: Schema-Level Input Validation

Instead of accepting a free-form command string, the schema should accept only structured, typed parameters that map to pre-approved operations. The filename parameter is constrained by a strict regex — only alphanumeric characters, hyphens, underscores, and a `.log` extension are accepted. `additionalProperties: false` ensures the agent cannot smuggle extra parameters through the schema boundary. The tool implementation constructs the full command server-side — the agent never touches the command string directly.

Layer 2: Server-Side Execution Policy Enforcement

Schema validation alone is insufficient because a sophisticated prompt injection attack might craft a filename that passes regex validation but exploits a downstream vulnerability. The MCP server must enforce: Path jail — resolve all file paths and verify they fall within the allowed directory using `os.path.realpath()`. Forbidden pattern scanner — scan the resolved command string for shell metacharacters before execution. Command whitelist — maintain an explicit list of permitted base commands. Output size cap — truncate all output at 150 lines to prevent data exfiltration.

Layer 3: Process Isolation and Privilege Separation

The MCP server process executing agent commands should never run as root. The principle of least privilege demands a dedicated low-privilege system user with read-only filesystem mounts outside the designated working directory, no network egress from the execution subprocess enforced via seccomp profiles, CPU and memory resource limits to prevent resource exhaustion attacks, and a short execution timeout with SIGKILL on breach.

Layer 4: Audit Logging and Anomaly Detection

Every MCP tool invocation must be logged with full fidelity: the agent's instruction context, exact input parameters, resolved command, execution result, and timestamp. An anomaly detection layer should monitor for repeated failed validation attempts within a short window, unusual filename patterns that are technically valid but semantically suspicious, burst invocation rates exceeding expected agent behavior baselines, and any invocation resulting in an execution policy rejection.

Architectural Trade-offs

Utility vs. Security: The more tightly constrained the schema, the less flexible the agent becomes. Teams must define explicit use case boundaries for each agent and design the sandbox to precisely fit that boundary. Static Schemas vs. Dynamic Capability Discovery: Prefer static, pre-declared tool manifests where the agent's tool access is fixed at session initialization. Synchronous Blocking vs. Async Review Queues: For high-risk operations, route the agent's request through an asynchronous human-review queue. Prompt Injection via Tool Outputs: Tool outputs should be explicitly labeled as data context, not instruction context, in the system prompt architecture.

Conclusion

MCP is a powerful protocol that unlocks genuinely transformative capabilities for autonomous developer agents. But power without constraint is a liability, not an asset. The engineering discipline of MCP sandboxing is not about limiting AI — it is about giving AI agents a clearly defined workspace where they can operate confidently without the risk of cascading unintended consequences. Zero-trust schema design, server-side execution policy enforcement, process isolation, and continuous audit logging together form a defense-in-depth architecture that makes autonomous agents both useful and safe.

Q22. Profile & Technical Portfolio Compilation

Professional Credentials & Portfolio Links

Full Name: Sonika Deshwal

University: Lovely Professional University

Registration Number: 12312987

LinkedIn: <https://www.linkedin.com/in/sonikadeshwal/>

GitHub: <https://github.com/sonikadeshwal>

LeetCode: https://leetcode.com/u/Sonika_Deshwal/

Resume

(PDF):

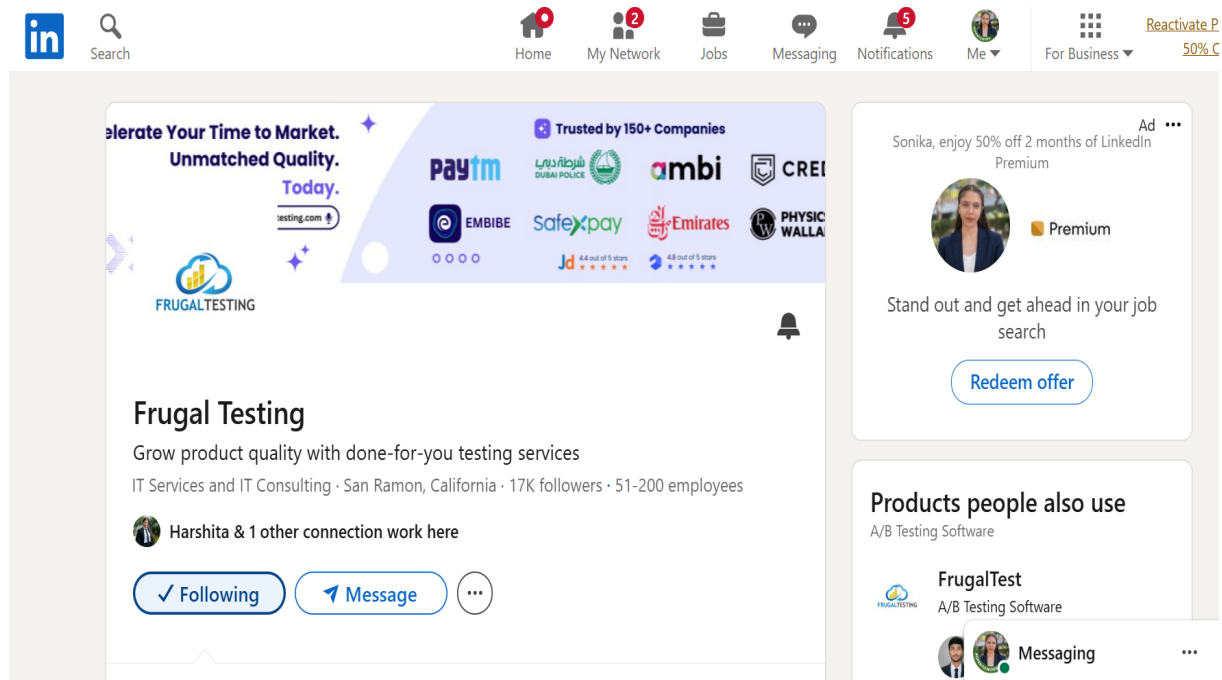
<https://drive.google.com/file/d/1KAp2j92FBXPxV4OWj-po3A4pRN91ViEF/view?usp=sharing>

Live Project: <https://fast-upload--sonikajaatdeshw.replit.app/>

Social Engagement — Proof of Follow

Followed and engaged with Frugal Testing on LinkedIn and YouTube as shown in screenshots below:

LinkedIn — Following Frugal Testing:



YouTube — Subscribed to FrugalTesting:

YouTube

Search

Q

+ Create

FRUGAL TESTING

NO BUGS IN SIGHT, OUR TESTING'S JUST RIGHT

FRUGAL TESTING

FrugalTesting

@frugaltesting • 2.83K subscribers • 106 videos

This is official Channel of FrugalTesting (<http://www.frugaltesting.com>). It is a self-service ...more

frugaltesting.com

Subscribed

Home

Videos

Shorts

Podcasts

Playlists

Q

FRUGAL TESTING

Frugal Testing | Perfect Quality in Every Digital Interaction | No...

2,526 views • 2 years ago

Q23. Video Evaluation Presentation (Video CV)

Google

Drive

Video

Link:

<https://drive.google.com/file/d/18E4efygE29zB5ij-EnoXQuu32th5UPfT/view?usp=sharing>

Duration: 2-3 minutes. The video covers:

1. How I redefine the role of an AI-Native Software Engineering professional
2. A complex technical challenge I solved — full-stack CRM with LLM integration and Vercel deployment debugging
3. How I balance generative AI tools without introducing security vulnerabilities or blind dependencies
4. How I would systematically approach a highly ambiguous technical problem without AI tools